Option C Task Report

COS30018 – Intelligent Systems
Lecturer: Dr. Nguyen Manh Toan
Group: 3.1 – OpenBuffet
Task name: C.6 – Machine Learning 2

For this week's task, we built on top of our Task C.5 code base and added an ensemble learning option to our stock prediction program. The main idea of this task is that instead of relying on just one model to make a prediction, we can combine the output of two different models and hope that their combination gives a more balanced and reliable result than either model alone. In our case we combined our existing deep learning model (GRU, but it also works with LSTM or RNN depending on the DL_NETWORK setting) with a classical statistical model, ARIMA, and also gave the option to combine it with a Random Forest Regressor instead, so the user can compare both **ensemble approaches**.

To implement ARIMA we used the statsmodels library in Python, specifically the **statsmodels.tsa.arima.model.ARIMA** class, since it is one of the most common and well documented libraries for classical time series forecasting and it was also recommended in the Medium article linked in the task sheet. For the Random Forest option we used the existing RandomForestRegressor class from sklearn.ensemble, which we had already used a bit in previous units.

Below are the detailed code descriptions of our work, focusing on the parts of the code that were not so straightforward and that required a bit of research on our part.

1. main.py / Interactive Menu – Option 6: Ensemble Learning Operations

The Task C.6 functionality was added as a new option (option 6) inside the interactive menu in main.py, which is the same menu that already lets the user pick between the Task C.4 and Task C.5 scenarios from previous weeks. Choosing option 6 first makes sure that the base GRU model from Task C.4 is loaded or trained (this is the model_base variable), because this is the deep learning half of the ensemble. Then the program asks the user a second question, this time whether they want to combine the deep learning model with ARIMA or with Random Forest.

```
if choice in ['1', '2', '6'] and model_base is None:
    ... load_or_build(...) the GRU model here ...
print("1. ARIMA")
print("2. Random Forest")
```

We reused model_base and predicted_prices_base rather than retraining the deep learning model again inside option 6. This was mainly to save time, since retraining a GRU model every single time we want to test a different ensemble combination would be quite wasteful, especially while we are still experimenting with different weights and settings.

2. main.py / ARIMA branch

```
train_prices_1d = train_data[PRICE_VALUE].values
arima_model = ARIMA(train_prices_1d, order=(5, 1, 0))
arima_fit = arima_model.fit()
ml_predictions = arima_fit.forecast(steps=len(test_data))
```

This is the part of the code that needed the most research on our side, since none of us had used the statsmodels library before this task. ARIMA stands for AutoRegressive Integrated Moving Average, and the order argument (5, 1, 0) sets its three main parameters, usually written as (p, d, q):

- p = 5: the number of lag observations used by the autoregressive part of the model.
- d = 1: the number of times the raw close prices are differenced to make the series more stationary. Stock prices generally trend upward over time, so differencing once helps remove that trend before the model tries to fit it.
- q = 0: the size of the moving average window, which we left at zero for a simpler model.

We picked (5, 1, 0) as a reasonably common starting configuration for daily stock data based on the statsmodels documentation and a few articles we found online about using ARIMA for financial time series, rather than doing a full grid search, since our main goal here was to get the ensemble pipeline working rather than to fine tune ARIMA on its own.

One thing that confused us at first is that ARIMA in statsmodels does not use the same fit/predict pattern as sklearn or Keras. Instead we call .fit() once on the training prices to get a fitted results object (arima_fit), and then call .forecast(steps=len(test_data)) on that object to generate a forecast for as many steps ahead as there are rows in the test set. This means ARIMA in our program is only ever trained on the training period and then forecasts forward blindly for the whole test period, unlike the deep learning model which slides a window across the test data one step at a time.

3. main.py / Random Forest branch

```
x_train_rf = x_train.reshape(x_train.shape[0], -1)
rf_model = RandomForestRegressor(n_estimators=150, max_depth=10, random_state=42)
rf_model.fit(x_train_rf, y_train)
rf_pred_scaled = rf_model.predict(x_test_rf).reshape(-1, 1)
ml_predictions = scaler.inverse_transform(rf_pred_scaled).flatten()
```

The Random Forest branch reuses the same x_train and y_train arrays that were originally built for the LSTM/GRU model, but Random Forest cannot take a 3-D array as input the way a recurrent layer can. This is why we call .reshape(x_train.shape[0], -1) first, which flattens each (PREDICTION_DAYS, 1) window into a single 1-D row of PREDICTION_DAYS values, keeping the number of samples the same but collapsing the timestep and feature dimensions into one. The same reshape is done on the test windows before calling .predict().

We also had to remember to inverse transform the Random Forest predictions using the same MinMaxScaler that was fitted on the training close prices, since the model itself only ever sees and outputs scaled values between 0 and 1. Forgetting this step gives predictions that look nothing like real prices, which is a mistake we actually made while first testing this part of the code.

4. main.py / Combining the predictions

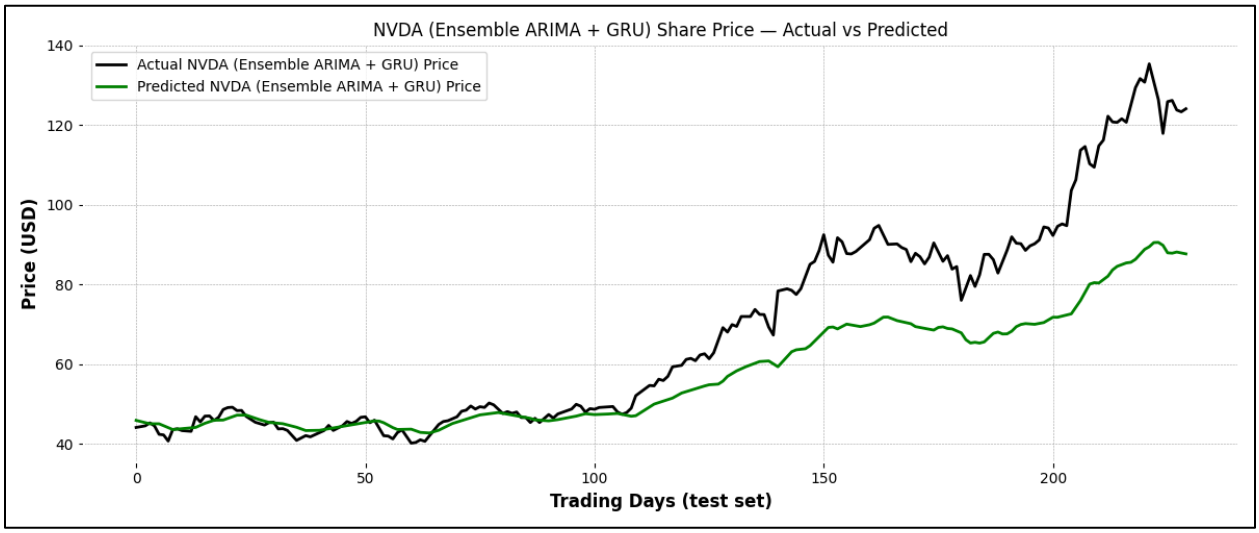
```
weight_ml = 0.4
        weight_dl = 0.6
        predicted_dl_flat = predicted_prices_base.flatten()
        ensemble_predicted_prices = (weight_ml * ml_predictions) + (weight_dl * predicted_dl_flat)
```

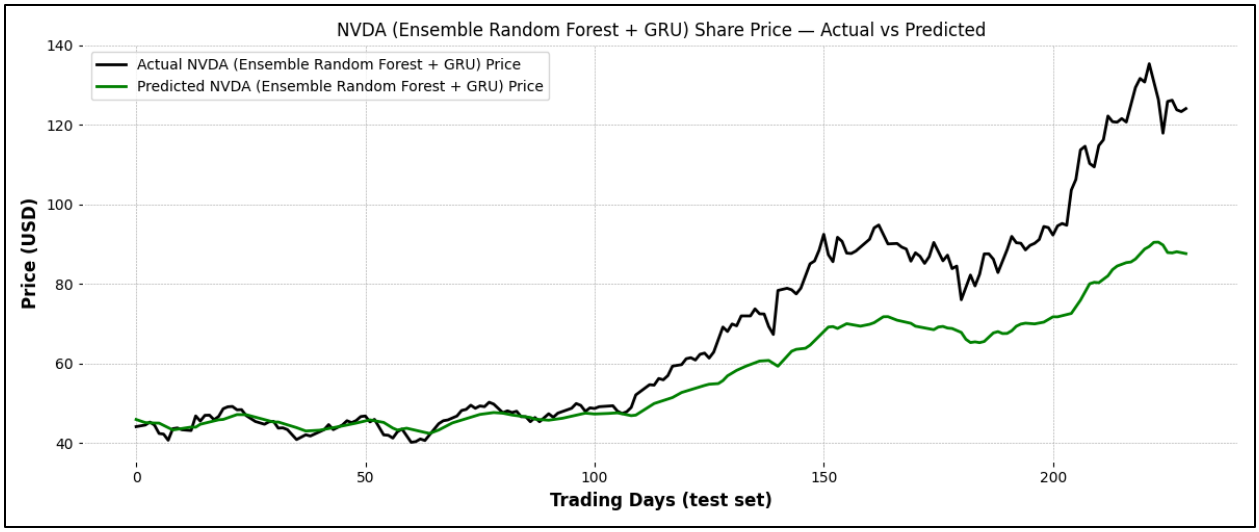
The ensemble itself is done with a simple weighted average of the two predictions, giving 60 percent weight to the deep learning model and 40 percent weight to whichever classical model was chosen. We went with a fixed weighting rather than something more complicated like a learned meta model, mainly because the task only asked for at least two models to be combined and we wanted to first make sure the simplest version of the idea worked correctly before adding more complexity. The weights were chosen by just trying a few combinations and eyeballing which one tracked the actual price line more closely on the plot, rather than through a formal search.

predicted_dl_flat is just predicted_prices_base.flatten(), which turns the (n, 1) column vector of GRU predictions into a plain 1-D array so it lines up with ml_predictions when we do the elementwise weighted sum above.

5. Experiment Results

Both ensemble combinations were tested on the same NVDA dataset used in previous tasks (2020-01-01 to 2024-07-02), with the same train/test split at 2023-08-02, and using GRU as the deep learning half of the ensemble.





6. Refactoring the Program into an Interactive Menu

Finally, since our program now has quite a few different prediction scenarios built up over the last few tasks (C.4 baseline prediction, C.5-1 multistep, C.5-2 multivariate, C.5-3 combined multivariate multistep, and now C.6 ensemble learning), we refactored main.py so that it does not just run through every scenario one after another every time the script is run. Instead, we added an interactive menu that lets the user type which prediction task they want to run, corresponding to each of the weekly tasks, and the program only builds or loads the model that is actually needed for that choice. This made it much easier for us to test one scenario at a time without waiting for every other model to train first, and it also makes the final program a lot more presentable, since anyone running it can just pick the option they are interested in from the menu instead of reading through the code to comment things in and out.